```
document.oncontextmenu = function(e){
            e.preventDefault()
            console.log('已经阻止了')
        }
```

默认行为

浏览器页面当中有很多行为，比如左击、右击、滚轮事件的行为。这些行为都不是自己添加上的，而是浏览器提供的。

```
document.oncontextmenu = function(e){
            e.preventDefault()
            console.log('已经阻止了')
        }
```

低版本IE浏览器的写法;

```
        e = e || window.event;
            e.returnValue = false
```

```
//直接用DOM零级去实现!
document.oncontextmenu = function(){
            console.log( 1 )
        return false;//不支持DOM二级的写法
}
```

在大多数情况下,为事件处理函数返回false,可以防止默认的事件行为.例如,默认情况下点击一个<a>元素,页面会跳转到该元素href属性指定的页。

js中return false作用一般是用来取消默认动作 / 阻止提交表单 / 阻止继续执行下面的代码。

监听写法

```
//主流浏览器的写法;可以阻止!
document.addEventListener('contextmenu',function(e){
    e.preventDefault();  //阻止默认行为的写法;
    console.log( '右键触发' )
});
```


```
//主流浏览器的写法;可以阻止!
document.addEventListener('contextmenu',function(e){
    e.preventDefault();  //阻止默认行为的写法;
    console.log( '右键触发' )
});
```

滚动事件与滚轮事件

滚动事件与滚轮事件的区别：

滚动事件是监听浏览器滚动条，只要浏览器的位置发生变化就可以发生变化。

滚轮事件是监听鼠标中键而不是轮动条；

滚动事件：

```
window.onscroll = function(){
    console.log( 'scroll' );
}
```

滚轮事件：

```
document.onmousewheel = function(){
    console.log( 'wheel' );
}
```

火狐浏览器

```
// 在非火狐浏览器的情况下，此值为null，
 // 在火狐浏览器的情况下，此值为undefined;
 console.log( window.onmousewheel ) //火狐浏览器没有这个属性，此值为undefined;

//火狐兼容写法：DOMMouseScroll
document.addEventListener( 'DOMMouseScroll' ,function(){
    console.log( 'DOMMouseScroll' );
});
```

滚动的方向

```
document.onmousewheel = function( e ){
          // 120及120的倍数;
    //120     向前推
           //-120    向后拉
          // 往下滚为负值，往上滚为正值;
          console.log( e.wheelDelta ) //滚动的步长;
      }

      document.addEventListener( "DOMMouseScroll" ,function(e){
           // console.log( 333 )
           //火狐浏览器滚动3或3的倍数;
    //3  向后拉
           // -3 向前推
           //往下滚为正数，  往上滚为负数;
           console.log( e.detail )
       })
```

封装滚轮事件

图片的放大缩小

见笔记html

```
<script>

    /*



    */

    // document.onmousewheel = function(  ){}

    //d:方向 e:event事件对象
    mousewheel( document ,function( d , e ){
        console.log( d , e )
        // console.log( 123 )
        // console.log( this )  // this要指向第一dom元素;
        return false; // dom零级的阻止方式;
    })

    function mousewheel( dom , fn ){
        // console.log( dom , fn )

        function summy(e){

            // console.log( '已经触发了' )
            // console.log( this )
            // console.log( e )

            //参一：改变fn的this指向;参二：统一值与统一方向; 第三个值

            if( fn.call( this, (e.wheelDelta/120 || -e.detail/3 ) , e ) === false ){

                //写阻止默认事件的方法;
                e.preventDefault ? e.preventDefault() : e.returnValue = false;

            }
            /*
                非火狐：e.wheelDelta (120/-120)/120  = 1
                火狐：e.detail  (-3/3) / -3 = 1
                统一：
                    向上滚：-1
                    向下滚：1
            */
            // return false; //阻止默认事件;

        }

        //兼容:火狐或非火狐的阻止;
        if( dom.onmousewheel === null ){

            //非火狐 : onmousewheel
            var type = "mousewheel"

        }else{

            //火狐 : DOMMouseScroll
            var type = "DOMMouseScroll"

        }


        //兼容：二级监听主流还是IE;
        if( document.addEventListener ){
            //主流
            dom.addEventListener( type ,summy,{ passive: false } )
        }else{
            //IE
            dom.attachEvent( "on"+ type ,summy )
        }

```

```
            // obj.onclick = function(){
            //     console.log( this )
            // }

        }

        // function fn(e ,d){
        //     console.log( e , d )
        //     console.log( this )
        // }
        // window.fn(1,3)


    </script>

</body>
```